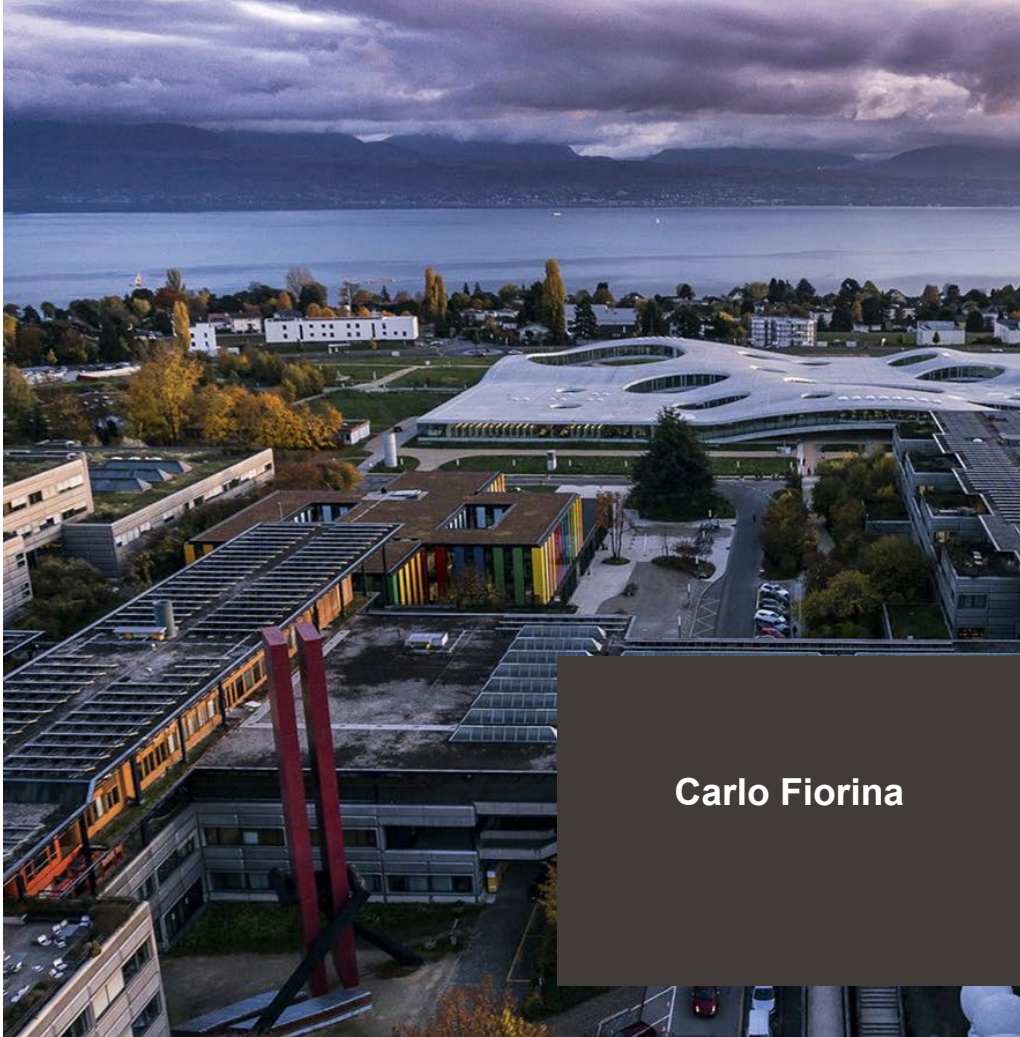




Use of OpenFOAM for multi- physics in nuclear

Carlo Fiorina

What to expect

- Overview of the modelling capabilities of OpenFOAM
- Lessons learnt
- A crash introduction to OFFBEAT and GeN-Foam

What not to expect

- A full course on the use of OpenFOAM
- A full course on the use of OFFBEAT
- A full course on the use of GeN-Foam

Objectives

- Understanding of modelling capabilities and pros & cons
- How to approach to OpenFOAM-based tools
- References/keywords/best practices for autonomous learning of GeN-Foam and OFFBEAT

Format:

- Presentations + interactive discussion
- 1 hands-on exercise

Content:

- Intro to modelling capabilities of OpenFOAM
- Intro to OFFBEAT
- Practical example based on OFFBEAT
- Intro to GeN-Foam

Material

- Slides distributed after workshop, incl. additional material from interactive discussion
- GeN-Foam available on GitLab
- OFFBEAT... Coming soon

 New codes from available numerical libraries



Open  FOAM

The Open Source CFD Toolbox

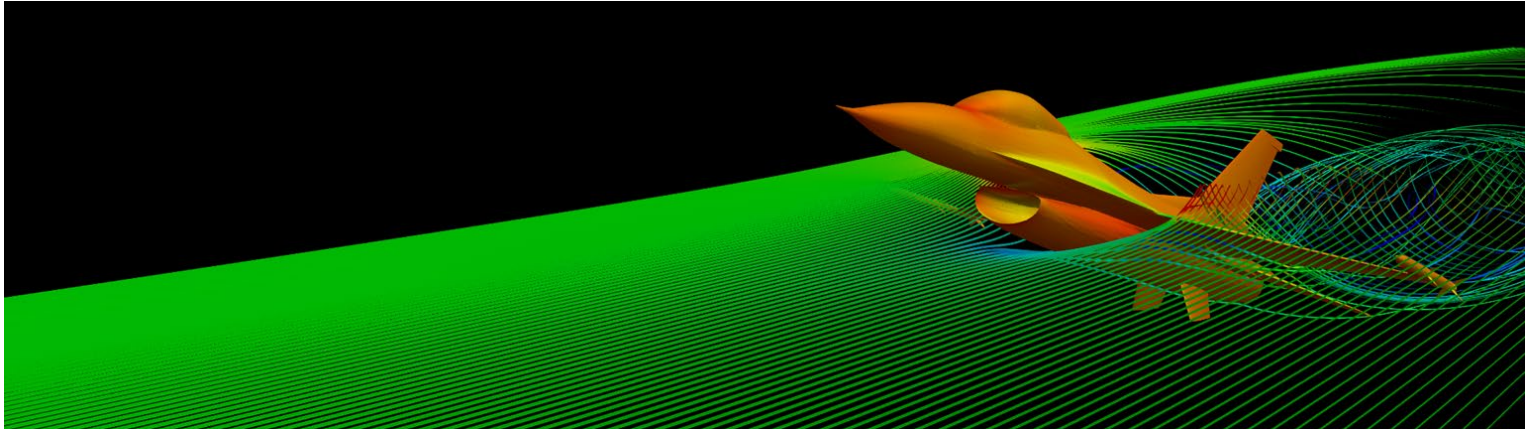
- Fast developments
- Modern algorithms
- Easier maintenance

❑ What is OpenFOAM?

- ✓ Distributed as CFD toolbox
- ✓ ~10k to 20k estimated users worldwide

Open  FOAM

The Open Source CFD Toolbox





The Open Source CFD Toolbox

□ What is OpenFOAM?

- ✓ Distributed as CFD toolbox
- ✓ ~10k to 20k estimated users worldwide
- ✓ OpenFOAM = Open Field Operation And Manipulation
- ✓ Essentially a large, well organized, HPC-scalable, C++ library for the finite-volume discretization and solution of PDEs, and including several functionalities like ODE solvers, projection algorithms, and mesh search algorithms
- ✓ Object-oriented, with a high-level “fail-safe” API

$$\frac{1}{v_i} \frac{\partial \varphi_i}{\partial t} - \Delta(D_i \varphi_i) = S$$

```
fvm::ddt(IV, flux_i)] - fvm::laplacian(D, flux_i)] = S
```

OpenFOAM: standing on the shoulders of giants

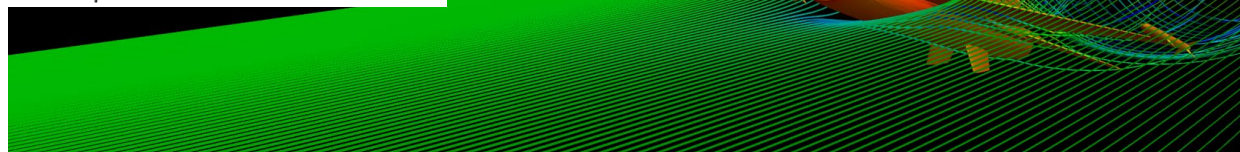
- ❑ CFD toolbox
 - ❑ Industrial level
- ❑ Library (90% of Gen-Foam and OFFBEAT)
 - ✓ Impressively well-written
 - ✓ Object-oriented
 - ✓ Complete (not only discretization and solution)
 - ✓ Quality assured
- ❑ Community (20k)
 - ✓ Concurrent developments
 - ✓ Support
- ❑ Drawbacks?
 - ✓ Workflow
 - ✓ Documentation

$$\frac{1}{v} \frac{\partial \varphi}{\partial t} = D \Delta \varphi + \left(\frac{v \Sigma_f * (1.0 - \beta_t) * \chi_p}{k_{eff}} - \Sigma_r \right) \varphi + S_d \chi_d$$

```
as
fvMatrix < scalar>
(
    fvm::ddt(inverseVelocity,flux)
    =fvm::laplacian(D, flux)
    +fvm::Sp(nuSigmaF/keff_*(1.0-betaTot)*chiPrompt      -      sigmaRe-
moval,flux)
    +delayedNeutronSource_*chiDelayed
);
```

Open  FOAM

The Open Source CFD Toolbox



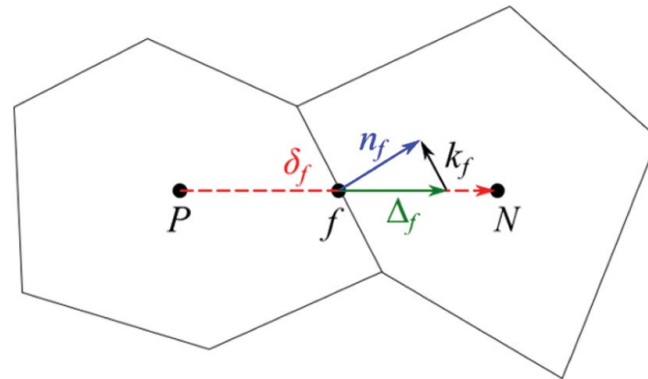
Conservation of quantities in each cell, with some wise interpolation at faces and techniques to deal with non-orthogonality and skewness

❑ Pros:

- ✓ Flexible
- ✓ Scalable
- ✓ Conservative
- ✓ CFD-friendly
- ✓ Intuitive

❑ Cons:

- ✓ Still require familiarity with concepts associated with PDEs (well-posed problems, initial and boundary conditions), geometry creation, meshing, discretization, linear solution, etc.
- ✓ Require good quality meshes
- ✓ Max second order in space





Most of the following is content taken from

- Carlo Fiorina, Ivor Clifford, Stephan Kelm, Stefano Lorenzi, 2022. “On the development of multi-physics tools for nuclear reactor analysis based on OpenFOAM[®]: state of the art, lessons learned and perspectives”. Nuclear Engineering and Design



Use of OpenFOAM for multi-physics

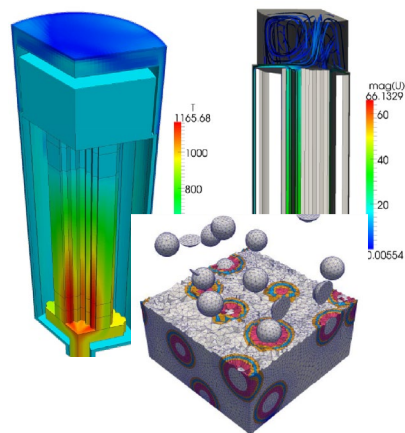
10

Carlo Fiorina

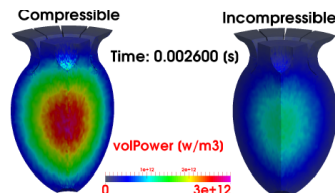
2000-2010
First activities

2010-2015
First widespread use

2015-2021
First coordinated and
persistent developments

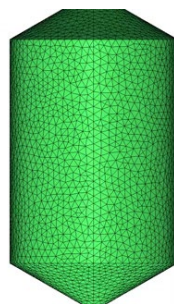


PBMRs and
HTRs

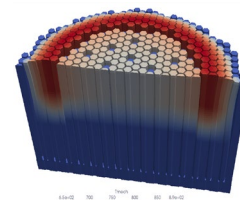
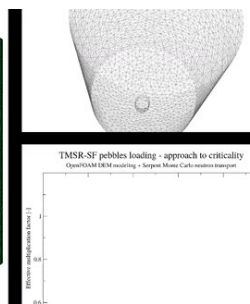


Temperature distribution

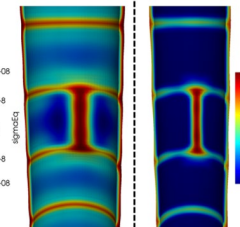
MSRs



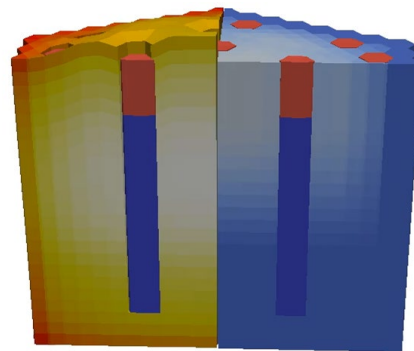
FHRs



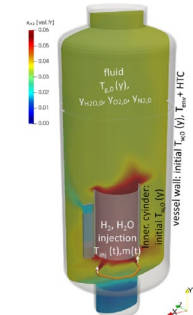
GeN-Foam



OFFBEAT

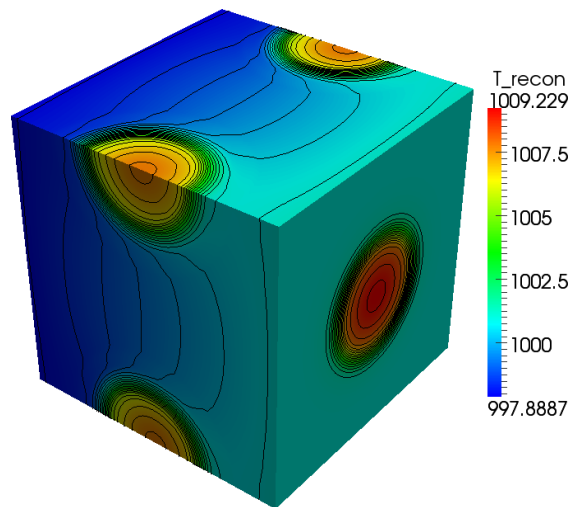


SFRs

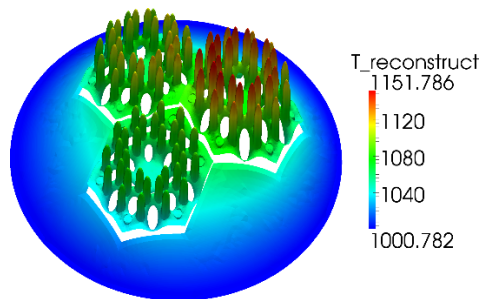


containmentFoam

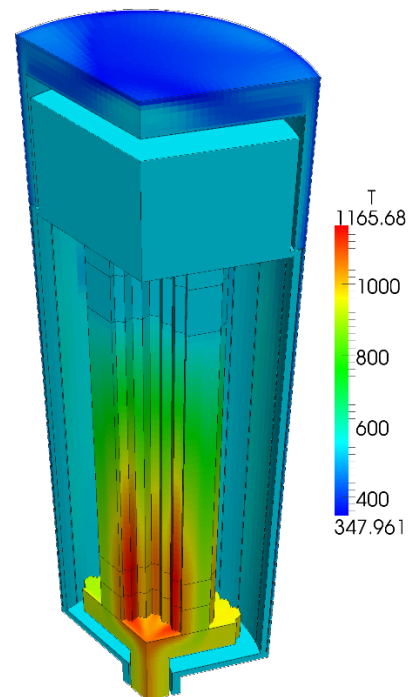
MHTGR-350MW Benchmark Model



*ROM reconstructed solution
for TRISO coated particles*



*ROM reconstructed
solution for a fuel
element*

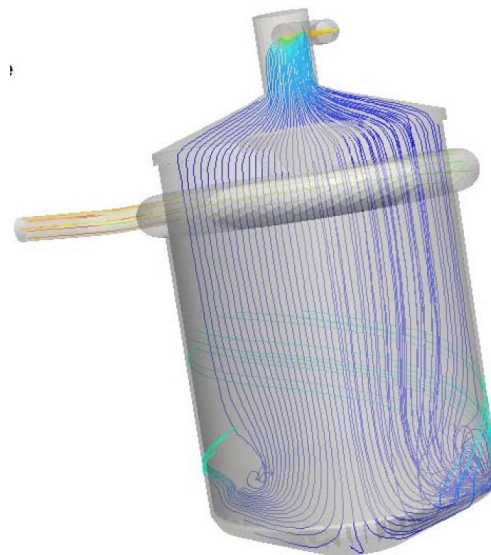
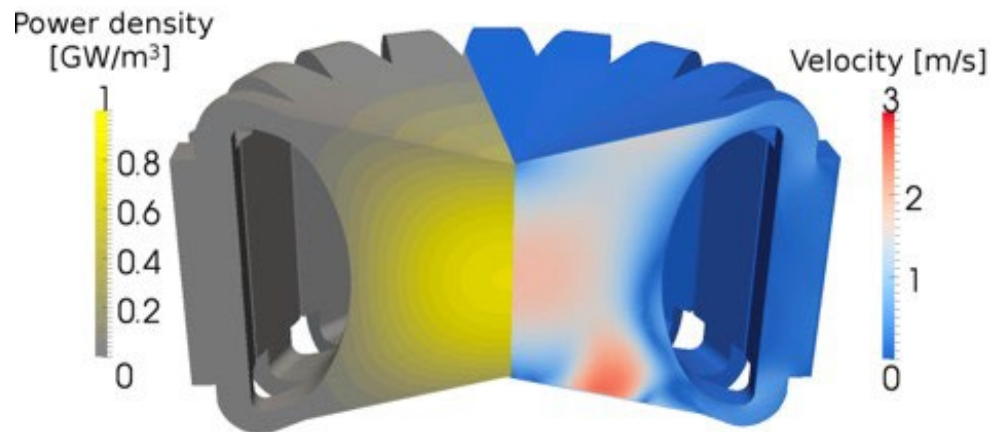


*Full-core coarse-mesh
thermal-hydraulics*

OpenFOAM -> sound TH solvers, ability to access underlying equation terms, customized equation operators for discontinuities, multiple arbitrary meshes, flexible boundary condition design (thermal radiation), classes for POD, time-integration using built-in ODE solvers,...

MSR modelling (M. Aufiero -> PolIMI + CNRS / GeN-Foam)

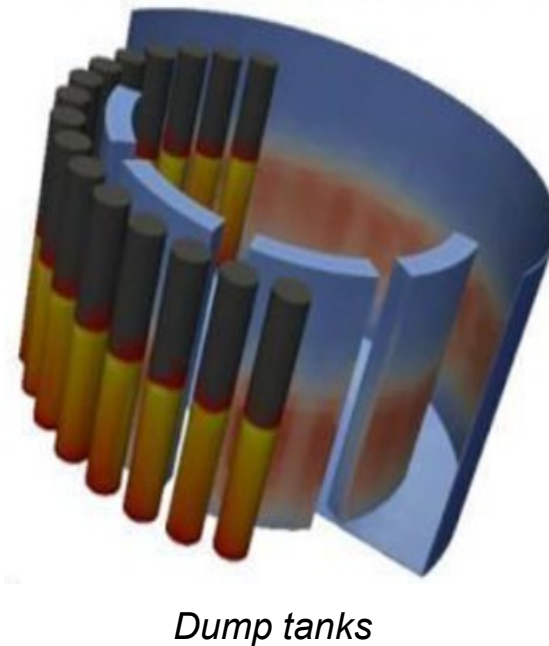
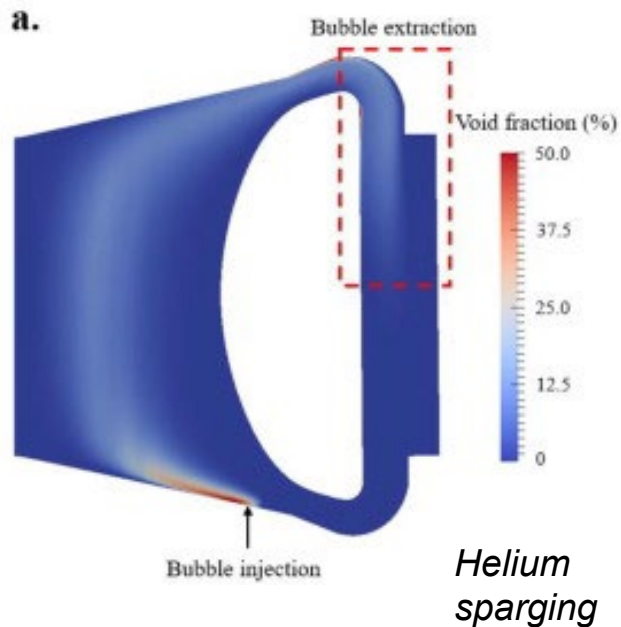
MSFR and MSRE



OpenFOAM -> Available CFD solvers, simple implementation of neutron diffusion and precursor transport, arbitrary geometries



MSR modelling: advanced

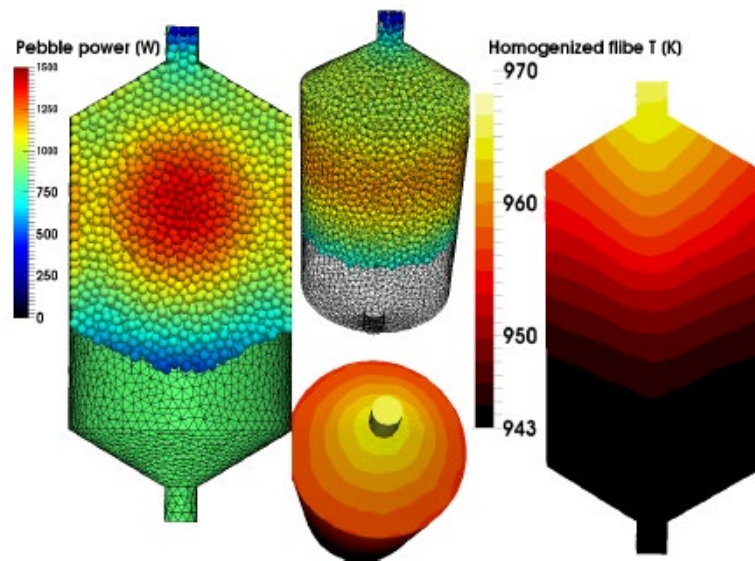


OpenFOAM -> Available two-phase CFD solvers, boundary conditions, radiative heat transfer

- Discrete Element Modeling + coarse-mesh thermal-hydraulics + Monte Carlo



*Helium
sparging*



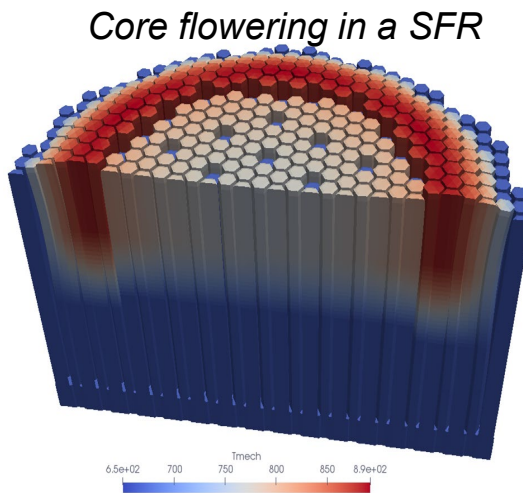
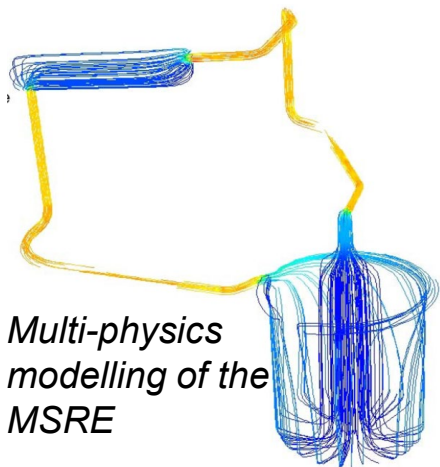
Dump tanks

OpenFOAM -> Available DEM solver, available multi-physics interface with Serpent, sound solvers (and discretization) for thermal-hydraulics

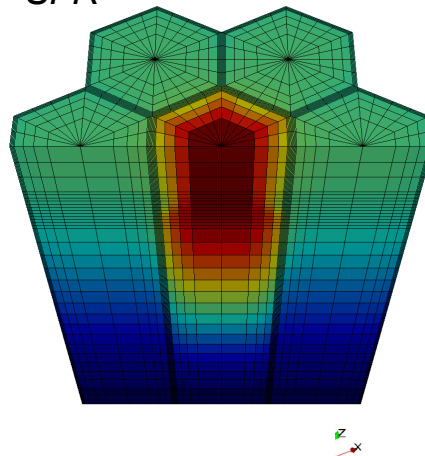


GeN-Foam: Generalized Nuclear Field operation and manipulation

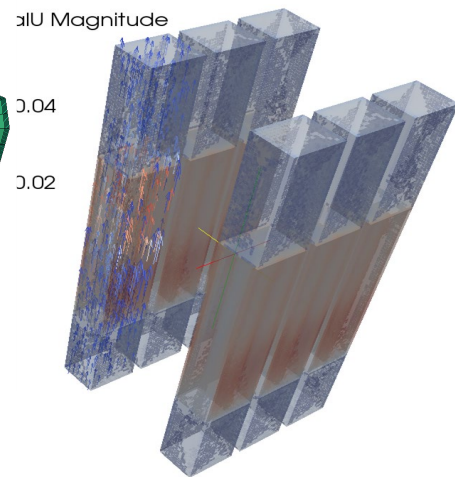
- First general solver for reactor safety based on OpenFOAM



Assembly windows in a SFR

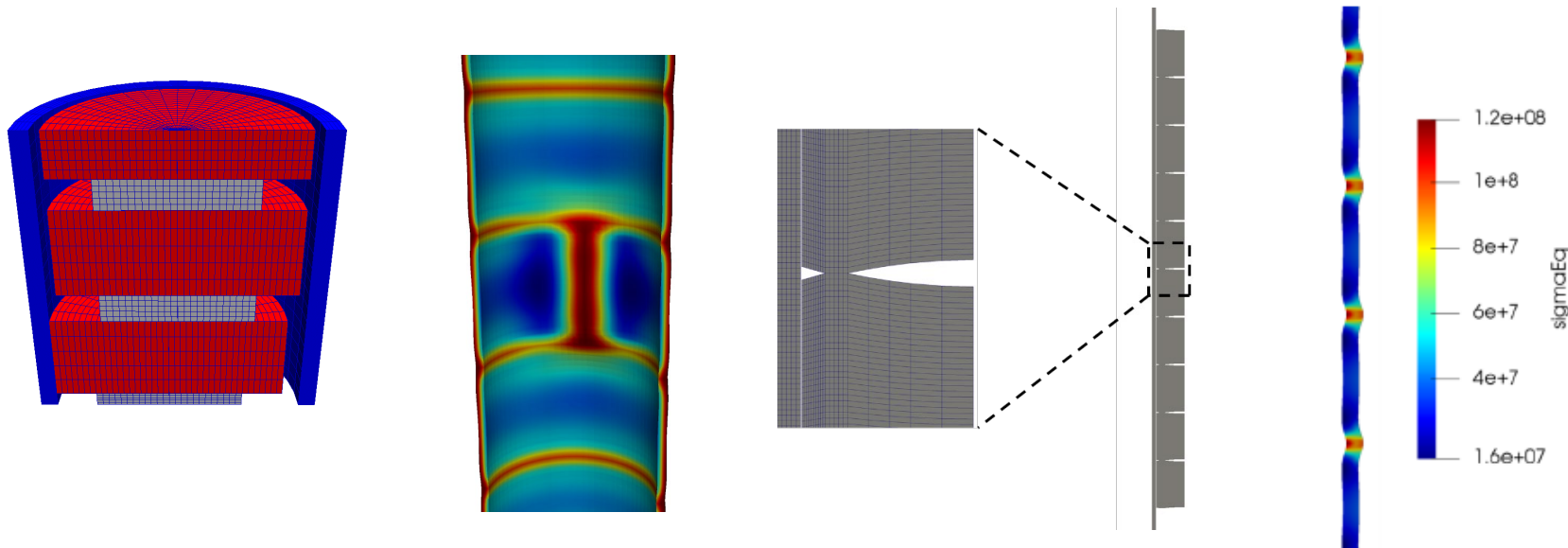


The Argonaut reactor



OpenFOAM -> Open-source + object oriented for making use of previous work, multi-mesh with projection algorithms, multi-material, ...

- Thermal-mechanics with finite volumes....

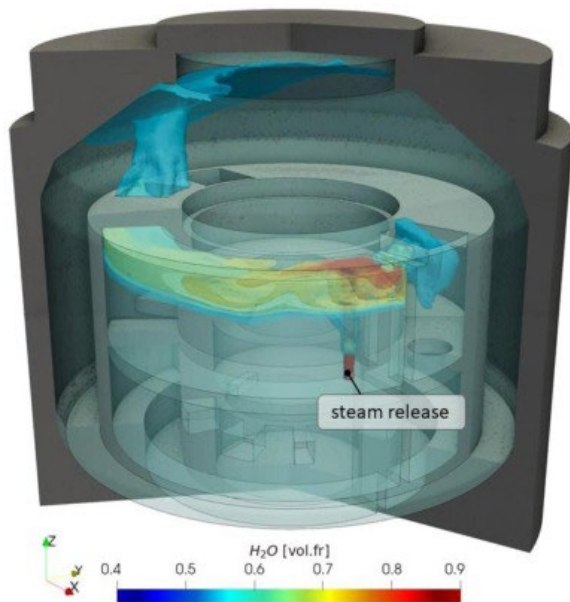


OpenFOAM -> community, intuitive formulation, conservative formulation, available functionalities (region-couple boundaries, ...), object-oriented library



HPC-oriented containment analysis

- High-fidelity solver for containment analysis (openly available!)



ISP-37 VANAM-M3 experiment
with containmentFOAM

OpenFOAM -> conservative formulation, available solvers (incl. Monte Carlo!), turbulent models, and available functionalities (region-couple boundaries, ...)

One can model pretty much everything...

What's the
effort?

What
competences
do I need?

What about the
license?

What is the
quality of the
result?

Downsides

- No graphical user interface (distributed with the code)
- Meshing and post-processing are performed with separate tools
- Meshing often requires proprietary tools
- Requires familiarity with Linux
- Limited documentation

Advantages

- Transparent
 - Access to source code
- 

Better integrations of application and development



Structure of the base library

Very complete

- discretization and linear system solution
- mesh-to-mesh projections
- mesh deformation
- mesh manipulation
- ordinary differential equations
- Monte Carlo methods
- octree-based mesh search
- methods for reduced-order modelling
- built-in and third-party code coupling schemes
- ...

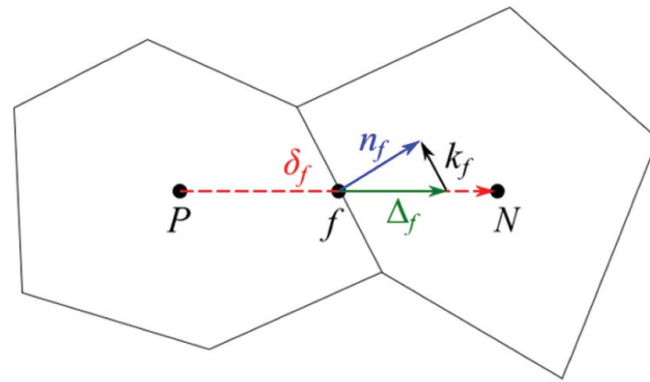
Object oriented

- encapsulation
- multi-level API

- Intuitive
- Conservative
- Optimal for thermal-hydraulics
- Good for thermal-mechanics
- Ok for neutronics

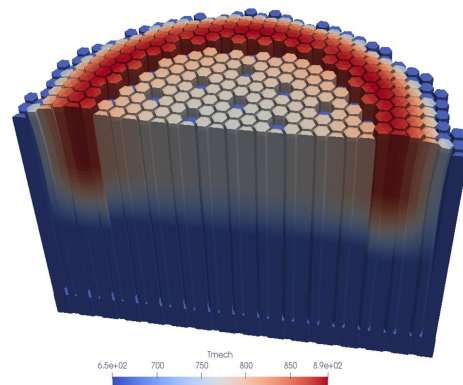
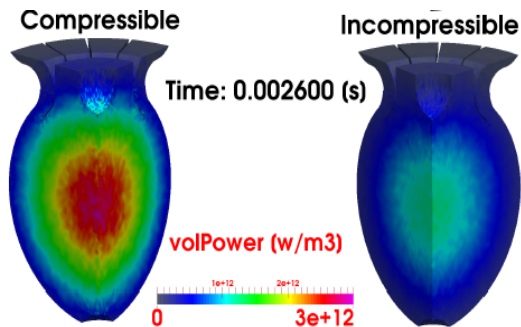
but

- Max second order
- Sensitive to mesh quality



Unstructured meshes

- Complete flexibility in terms of geometry -> non-traditional reactor designs and complex component
- Significant computational footprint
- First order, with all cell faces that are flat -> a high mesh resolution for curved surfaces



Operator-splitting

One matrix for each equation + iteration

Pros

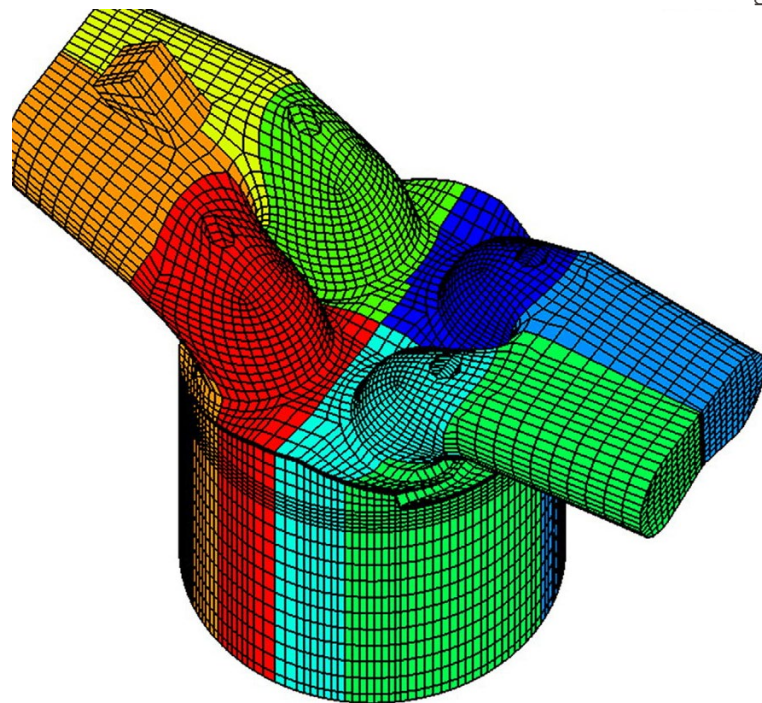
- Easier preconditioning and optimal choice of solution method
- No need to solve all physics at each coupling/time step

Cons

- Can be hard to converge for weakly-coupled / strongly non-linear equations



- Domain decomposition and the MPI
- Optimally scale up to few thousands of CPU cores
- Some bottlenecks
 - ✓ the sub-optimal sparse matrices storage format (LDU) that does not enable any cache-blocking mechanism (SIMD, vectorization)
 - ✓ the I/O data storage system
- The OpenFOAM HPC Technical Committee is currently working on the limitations
 - ✓ interface to external linear algebra libraries
 - ✓ recent work from NVIDIA





Computational requirements

CPU cores

- rule of thumb: 30'000 mesh cells per CPU core
- CFD
 - 2D RANS-> several hundred thousand cells -> 10 CPU cores
 - 3D RANS -> several hundred millions cells -> 5000 CPU cores
- coarse-mesh thermal-hydraulics and neutron diffusion
 - full-core models -> few hundred thousand to few million cells -> workstations or laptops

Runtime

- Steady-state simulations on the optimal number of CPU cores: several minutes to several hours
- Long-running time-dependent problems: up to a week
- In some specific applications, such as detailed containment simulations: up to a month

Memory requirements

- Single-phase RANS CFD simulation -> order of 10 fields -> 1 GB of memory per million cells
- 3D discrete ordinates neutron transport -> several thousand solution fields -> 200 GB of memory per million cells

- GNU GPLv3 license
 - copyleft type license: automatically affect derivative work
 - favors a collaborative development with minimal work duplication
 - limits investments from commercial players



Take away messages

C. Fiorina

28

Carlo Fiorina

- With some practice and commitment OpenFOAM allows one to model pretty much everything, ending up with a solver featuring state-of-the-art numerics
- No need to start from scratch:
 - Many developers will share their work
 - Some solvers are already available online
- Not necessarily an easy tool: Make sure you or your students/employees are relatively familiar with CFD methods, or with numerical methods for PDEs (possibly finite volumes)
- Don't get frustrated: there is always a way out with OpenFOAM and, most likely, someone who had your same problem and will be happy to help
- Don't get discouraged: the entry barrier may seem steep, but skills one learns will allow them to tackle many problems
- If possible, do not do it alone!

**Thank
you**

Carlo Fiorina



Porous-medium thermal-hydraulics: volume averaging

$$\frac{\partial}{\partial t} \rho_i^* + \nabla \cdot (\mathbf{u}_i^* \rho_i^*) = 0$$

$$\langle \phi_i^* \rangle_I = \frac{1}{V_i} \int_{\Omega_i} \phi_i^* dV$$

$$\langle \phi_i^* \rangle = \frac{1}{V} \int_{\Omega_i} \phi_i^* dV$$

$$\frac{\partial}{\partial t} (\alpha_i \rho_i) + \nabla \cdot (\alpha_i \mathbf{u}_i \rho_i) = \sum_{j \neq i} \frac{1}{V} \int_{\partial \Omega_{ij}} \rho_i^* (\mathbf{u}_\partial^* - \mathbf{u}_i^*) \cdot \mathbf{n} dS$$

Volume averaging results in:

- additional **variables** (phase fraction, tortuosity, etc.);
- additional terms that require **experimentally-informed closure**;



Porous-medium thermal-hydraulics: Governing equations

The coarse-mesh governing equations (Navier-Stokes and enthalpy) are:

$$\frac{\partial}{\partial t} (\alpha_i \rho_i) + \nabla \cdot (\alpha_i \mathbf{u}_i \rho_i) = -\Gamma_{i \rightarrow j}$$

$$\frac{\partial}{\partial t} (\alpha_i \rho_i \mathbf{u}_i) + \nabla \cdot (\alpha_i \rho_i \mathbf{u}_i \otimes \mathbf{u}_i) =$$

$$- \alpha_i \nabla p + \nabla \cdot (\alpha_i \boldsymbol{\sigma}_{d,i}) + \alpha_i \rho_i \mathbf{g} - \mathbf{S}_{\mathbf{u},i \rightarrow j}$$

$$\frac{\partial}{\partial t} (\alpha_i \rho_i h_i) + \nabla \cdot (\alpha_i \mathbf{u}_i \rho_i h_i) =$$

$$\nabla \cdot (\alpha_i \kappa_i \mathbf{T}_i \cdot \nabla T_i) + \alpha_i \frac{\partial}{\partial t} p + \alpha_i \rho_i \mathbf{u}_i \cdot \mathbf{g} + \alpha_i q_{int,i} - S_{h,i \rightarrow j}$$

These reduce to traditional CFD approaches in clear fluid regions and a system-code-like approach in 1-D regions (multiple scales).